Gemini

New chat

Search chats

Traceback (most recent call last): File "...

Django Import Fixer Script

VS Code Regex Exclude Import

Cherry Servers Infrastructure Recomm...

Vue3 "Not Iterable" Error Fix

Missing SNV-GMidos setting

Windows Proxmox Ethernet Network S...

Nginx Custom Build Crashing During S...

Checking Ubuntu Swap Status

Django Unfold Server Component Admin

Cherry Servers VPN Infrastructure Inqu...

Diagnosing Network Issues with Tracer...

Docker Secrets Management Open So...

Netplan Configuration Analysis and Ex...

Floating IP for OPNsense Firewall

Codebase Cleanup Scripts for Sale

Fix Invoice Billing Term Logic

Vue 3 Reactivity Fix for PayPal Button

Fixing Django Invoice Payment Logic

Refactoring Hourly Billing Integration

Debugging Hourly Billing Integration Is...

Upgrade

Please help to create a robust bash script that I will run in an Linux Ubuntu rescue environment that will do the following.

When it executes it will ask to enter 2 serial numbers fo

Here is a robust bash script designed for an Ubuntu rescue environment.

Since you are installing Windows on an EFI/NVMe system, standard UEFI requires the bootable disk to be formatted as FAT32. A major hurdle when doing this manually in Linux is that modern Windows `install.wim` files are often larger than 4GB, which exceeds the FAT32 file size limit.

To solve this, the script automatically installs `wimtools` and `p7zip-full`, formats the ISO disk as FAT32, extracts the ISO, and actively splits the `.wim` file if it breaches the 4GB limit so that your motherboard UEFI can successfully boot the Windows installer.

The Bootstrap Script

Save this script as `prep_windows.sh`, make it executable (`chmod +x prep_windows.sh`), and run it as root.

Bash

```
#!/usr/bin/env bash
set -euo pipefail

# Ensure the script is run as root
if [[ $EUID -ne 0 ]]; then
   echo "Error: This script must be run as root."
   exit 1
fi

echo "Updating repositories and installing required packages..."
apt-get update -y
apt-get install -y p7zip-full wimtools parted dosfstools util-linux

# 1. Prompt for the two NVMe serial numbers
echo -n "Enter 2 NVMe serial numbers separated by a space: "
read -r SERIAL1 SERIAL2

# 2. Prompt for the ISO path
echo -n "Enter absolute path to bootable Windows ISO: "
read -r ISO_PATH

if [[ ! -f "$ISO_PATH" ]]; then
    echo "Error: ISO file not found at $ISO_PATH"
    exit 1
fi

# 3. Prompt for the destination NVMe drive
echo -n "Enter the destination NVMe serial (for the Windows bootloaders): "
read -r DEST_SERIAL

# Function to map serial numbers to NVMe block devices
get_dev_by_serial() {
    local target_serial="$1"
    # Read lsblk output. $1 is NAME, $2 is SERIAL
    lsblk -ndo NAME,SERIAL | while read -r dev sn; do
        if [[ "$sn" == "$target_serial" ]]; then
            echo "/dev/$dev"
            return
        fi
    done
}

DEV1=$(get_dev_by_serial "$SERIAL1")
DEV2=$(get_dev_by_serial "$SERIAL2")

if [[ -z "$DEV1" ]] || [[ -z "$DEV2" ]]; then
    echo "Error: Could not locate NVMe drives matching one or both provided serial n
    exit 1
fi

# Identify the Target (Bootloader) disk and the ISO (Installer) disk
if [[ "$DEST_SERIAL" == "$SERIAL1" ]]; then
    TARGET_DEV="$DEV1"
    ISO_DEV="$DEV2"
elif [[ "$DEST_SERIAL" == "$SERIAL2" ]]; then
    TARGET_DEV="$DEV2"
    ISO_DEV="$DEV1"
else
    echo "Error: Destination serial must exactly match one of the two initial seri
    exit 1
fi

echo "-----------------------------------------------------"
echo "Target Bootloader Disk : $TARGET_DEV ($DEST_SERIAL)"
echo "ISO Installation Disk  : $ISO_DEV (Other)"
echo "Windows ISO Path       : $ISO_PATH"
echo "-----------------------------------------------------"
echo "WARNING: Both disks will be COMPLETELY WIPED."
echo -n "Type 'YES' to proceed: "
read -r CONFIRM
if [[ "$CONFIRM" != "YES" ]]; then
    echo "Operation aborted."
    exit 0
fi

# Setup cleanup trap for mounts
MNT_ISO_DISK=$(mktemp -d)
cleanup() {
    echo "Cleaning up mounts..."
    umount "$MNT_ISO_DISK" 2>/dev/null || true
    rmdir "$MNT_ISO_DISK" 2>/dev/null || true
}
trap cleanup EXIT

# -----------------------------------------------------
# PREPARE TARGET DISK (Destination for Windows OS/Bootloader)
# -----------------------------------------------------
echo "Preparing Target Disk ($TARGET_DEV)..."
wipefs -a "$TARGET_DEV"
parted -s "$TARGET_DEV" mklabel gpt
# Create an empty EFI System Partition (ESP) so the Windows Installer
# immediately recognizes this drive as a valid EFI boot destination.
parted -s "$TARGET_DEV" mkpart EFI fat32 1MiB 513MiB
parted -s "$TARGET_DEV" set 1 esp on
sleep 2 # Allow udev to catch up
TARGET_PART="${TARGET_DEV}p1"
mkfs.fat -F32 "$TARGET_PART"

# -----------------------------------------------------
# PREPARE ISO DISK (Acting as the Windows Installation USB)
# -----------------------------------------------------
echo "Preparing ISO Disk ($ISO_DEV)..."
wipefs -a "$ISO_DEV"
parted -s "$ISO_DEV" mklabel gpt
parted -s "$ISO_DEV" mkpart primary fat32 1MiB 100%
parted -s "$ISO_DEV" set 1 esp on
sleep 2 # Allow udev to catch up
ISO_PART="${ISO_DEV}p1"

echo "Formatting ISO Disk as FAT32..."
mkfs.fat -F32 "$ISO_PART"

echo "Mounting ISO Disk..."
mount "$ISO_PART" "$MNT_ISO_DISK"

echo "Extracting Windows ISO to $ISO_DEV..."
7z x "$ISO_PATH" -o"$MNT_ISO_DISK"

# Handle the 4GB FAT32 limit for install.wim
# Using a case-insensitive search because 7z extraction casing can vary
echo "Checking for oversized install.wim..."
WIM_FILE=$(find "$MNT_ISO_DISK" -type f -iregex '.*/sources/install\.wim$')

if [[ -n "$WIM_FILE" ]]; then
    WIM_SIZE=$(stat -c%s "$WIM_FILE")
    # 4GB limit is 4294967295 bytes
    if [[ $WIM_SIZE -gt 4294967295 ]]; then
        echo "install.wim is larger than 4GB. Splitting into install.swm files..."
        WIM_DIR=$(dirname "$WIM_FILE")
        # Split into 3800MB chunks
        wimlib-imagex split "$WIM_FILE" "$WIM_DIR/install.swm" 3800
        echo "Removing original oversized install.wim..."
        rm -f "$WIM_FILE"
    else
        echo "install.wim is under 4GB. No splitting required."
    fi
else
    echo "No install.wim found (might be an install.esd). Proceeding."
fi

# Ensure disk buffers are flushed
sync

echo "====================================================="
echo "SUCCESS! Disks prepared."
echo "You can now reboot. Ensure your motherboard UEFI is set "
echo "to boot from the ISO Disk ($ISO_DEV)."
echo "====================================================="
```

What this achieves:

1. **Target Disk Isolation:** It creates a clean GPT partition table and a blank EFI System Partition on your destination drive. This guarantees that when the Windows installer boots, it has a valid target for the Windows Boot Manager (`bootmgfw.efi`).
2. **ISO Disk Formatting:** It builds out a perfectly structured FAT32 drive for standard UEFI boot and unpacks the ISO there.
3. chunking out the WIM file securely. Standard FAT32 will crash on transfer of a file > 4GB, which practically every Windows 10/11 ISO contains.

Please explain what should I do exactly, should I first mount the disk which the ISO will be written to where do I run the wget command to download the iso and the next steps, please list the commands in order

First, a quick observation from your `lsblk` output: your system is using standard SATA/SAS SSDs (`sda` and `sdb`), not NVMe drives (`nvme0n1`). **This is perfectly fine.** The script is designed to search by the serial number, so it will automatically find `sda` and `sdb` just as easily.

Because the script completely wipes both disks, **you cannot download the ISO directly to `sda` or `sdb`**. If you did, the script would delete the ISO before it had a chance to read it. Instead, you will download the ISO directly into the rescue environment's RAM (usually mounted at `/tmp`).

Here is the exact order of operations. You do not need to mount anything yourself; the script handles all of that.

1. **Identify your serial numbers**

   You need these for the script prompts

   Run this command to view your drives and their serial numbers:

   ```
   lsblk -o NAME,SERIAL,SIZE
   ```

   Copy the serial number for `sda` and the serial number for `sdb`. You will need them exactly as they appear.

2. **Download the ISO into RAM**

   Requires at least 8GB of server RAM

   Navigate to the `/tmp` directory (which is stored in your server's memory) and download the ISO using `wget`.

   Note: Replace the URL in quotes with your actual direct download link to the Windows ISO.

   ```
   cd /tmp
   wget -O windows.iso "https://your-domain.com/path/to/windows.iso"
   ```

3. **Create and save the script**

   Open a new file using the `nano` text editor:

   ```
   nano prep_windows.sh
   ```

   1. Paste the entire script from my previous message into the editor.
   2. Press **Ctrl+O** (the letter O) and hit **Enter** to save.
   3. Press **Ctrl+X** to exit `nano`.

   Now, make the script executable:

   ```
   chmod +x prep_windows.sh
   ```

4. **Execute the script**

   Run the script:

   ```
   ./prep_windows.sh
   ```

   When prompted, answer exactly like this:

   1. **Enter 2 NVMe serial numbers:** Paste your two serials separated by a space (e.g., `SERIAL_SDA SERIAL_SDB`) and press Enter.
   2. **Enter absolute path to bootable Windows ISO:** Type `/tmp/windows.iso` and press Enter.
   3. **Enter the destination NVMe serial:** Paste the serial number of the drive where you want Windows permanently installed (e.g., your `sda` serial) and press Enter.
   4. Type YES to confirm the wipe.

Once the script finishes and prints the success message, type `reboot` in your terminal. You will then need to access your server's KVM/IPMI console to watch the boot process and ensure it boots from the ISO disk to begin the Windows installation.

This is the part I am unsure about, the reason I ask is I want Windows to detect all installation files

root@175312:~# lsblk -o NAME,SERIAL,SIZE

You are right at the final step! Since you have two identical 232.9G drives, it doesn't strictly matter which one you pick as the destination — but let's use `sda` as your main Windows OS drive.

Copy and paste this exact serial number into your terminal prompt right now and press Enter:

`S4CJNF0MA20067T`

Why this works (and why Windows will find everything)

When you give the script this specific serial number, it assigns the roles perfectly:

1. **The Destination (`sda`):** It preps `sda` with a clean partition table so the Windows installer immediately recognizes it as a valid, blank target to install the operating system onto.
2. **The Installer (`sdb`):** It automatically selects your *other* drive (`sdb` / `S4CJNZFN204309M`) to act as your "bootable USB drive." It formats `sdb` as FAT32, extracts all the files from your Windows Server 2019 ISO onto it, and handles any files larger than 4GB.

Because the installation files are fully extracted onto `sdb`, your motherboard will boot directly from it. Once the Windows setup screen loads, it will naturally detect `sda` as the empty drive waiting for the installation.

After you press Enter, the script will ask for a final confirmation. Type `YES` and press Enter again. Once it finishes and prints the success message, type `reboot` and connect to your server's KVM/IPMI console to walk through the Windows setup!

No containers need to be restarted.

No user sessions are running outdated binaries.

I see exactly what happened! This was my oversight.

Earlier, we noticed your server is using SATA/SAS SSDs (`sda` and `sdb`) instead of NVMe drives, but the script still assumed the NVMe partition naming convention.

In Linux, when you partition an NVMe drive, it adds a `p` before the partition number (e.g., `/dev/nvme0n1` becomes `/dev/nvme0n1p1`). But for standard SATA drives, it just slaps the number directly on the end (e.g., `/dev/sda` becomes `/dev/sda1`, not `/dev/sdap1`).

The script failed because it tried to format `/dev/sdap1`, which doesn't exist.

Here is the corrected script that intelligently checks the drive name and applies the correct partition naming rule.

How to fix it

Open the file again and delete the old contents:

```
nano prep_windows.sh
```

(Tip: In nano, you can hold **Ctrl+K** *to quickly cut/delete all the lines)*

Paste this updated version in:

```
#!/usr/bin/env bash
set -euo pipefail

if [[ $EUID -ne 0 ]]; then
   echo "Error: This script must be run as root."
   exit 1
fi

echo "Updating repositories and installing required packages..."
apt-get update -y
apt-get install -y p7zip-full wimtools parted dosfstools util-linux

echo -n "Enter 2 NVMe/SATA serial numbers separated by a space: "
read -r SERIAL1 SERIAL2

echo -n "Enter absolute path to bootable Windows ISO: "
read -r ISO_PATH

if [[ ! -f "$ISO_PATH" ]]; then
    echo "Error: ISO file not found at $ISO_PATH"
    exit 1
fi

echo -n "Enter the destination drive serial (for the Windows bootloaders): "
read -r DEST_SERIAL

get_dev_by_serial() {
    local target_serial="$1"
    lsblk -ndo NAME,SERIAL | while read -r dev sn; do
        if [[ "$sn" == "$target_serial" ]]; then
            echo "/dev/$dev"
            return
        fi
    done
}

DEV1=$(get_dev_by_serial "$SERIAL1")
DEV2=$(get_dev_by_serial "$SERIAL2")

if [[ -z "$DEV1" ]] || [[ -z "$DEV2" ]]; then
    echo "Error: Could not locate drives matching one or both provided serial numbers
    exit 1
fi

if [[ "$DEST_SERIAL" == "$SERIAL1" ]]; then
    TARGET_DEV="$DEV1"
    ISO_DEV="$DEV2"
elif [[ "$DEST_SERIAL" == "$SERIAL2" ]]; then
    TARGET_DEV="$DEV2"
    ISO_DEV="$DEV1"
else
    echo "Error: Destination serial must exactly match one of the two initial seri
    exit 1
fi

# Helper function to handle NVMe (p1) vs SATA (1) partition naming
get_part_name() {
    if [[ "$1" == *[0-9] ]]; then
        echo "${1}p1" # NVMe style: /dev/nvme0n1 -> /dev/nvme0n1p1
    else
        echo "${1}1"  # SATA style: /dev/sda -> /dev/sda1
    fi
}

echo "-----------------------------------------------------"
echo "Target Bootloader Disk : $TARGET_DEV ($DEST_SERIAL)"
echo "ISO Installation Disk  : $ISO_DEV (Other)"
echo "Windows ISO Path       : $ISO_PATH"
echo "-----------------------------------------------------"
echo "WARNING: Both disks will be COMPLETELY WIPED."
echo -n "Type 'YES' to proceed: "
read -r CONFIRM
if [[ "$CONFIRM" != "YES" ]]; then
    echo "Operation aborted."
    exit 0
fi

MNT_ISO_DISK=$(mktemp -d)
cleanup() {
    echo "Cleaning up mounts..."
    umount "$MNT_ISO_DISK" 2>/dev/null || true
    rmdir "$MNT_ISO_DISK" 2>/dev/null || true
}
trap cleanup EXIT

# -----------------------------------------------------
# PREPARE TARGET DISK
# -----------------------------------------------------
echo "Preparing Target Disk ($TARGET_DEV)..."
wipefs -a "$TARGET_DEV"
parted -s "$TARGET_DEV" mklabel gpt
parted -s "$TARGET_DEV" mkpart EFI fat32 1MiB 513MiB
parted -s "$TARGET_DEV" set 1 esp on
sleep 2

TARGET_PART=$(get_part_name "$TARGET_DEV")
echo "Formatting Target ESP on $TARGET_PART..."
mkfs.fat -F32 "$TARGET_PART"

# -----------------------------------------------------
# PREPARE ISO DISK
# -----------------------------------------------------
echo "Preparing ISO Disk ($ISO_DEV)..."
wipefs -a "$ISO_DEV"
parted -s "$ISO_DEV" mklabel gpt
parted -s "$ISO_DEV" mkpart primary fat32 1MiB 100%
parted -s "$ISO_DEV" set 1 esp on
sleep 2

ISO_PART=$(get_part_name "$ISO_DEV")
echo "Formatting ISO Disk as FAT32 on $ISO_PART..."
mkfs.fat -F32 "$ISO_PART"

echo "Mounting ISO Disk..."
mount "$ISO_PART" "$MNT_ISO_DISK"

echo "Extracting Windows ISO to $ISO_DEV..."
7z x "$ISO_PATH" -o"$MNT_ISO_DISK"

echo "Checking for oversized install.wim..."
WIM_FILE=$(find "$MNT_ISO_DISK" -type f -iregex '.*/sources/install\.wim$')

if [[ -n "$WIM_FILE" ]]; then
    WIM_SIZE=$(stat -c%s "$WIM_FILE")
    if [[ $WIM_SIZE -gt 4294967295 ]]; then
        echo "install.wim is larger than 4GB. Splitting into install.swm files..."
        WIM_DIR=$(dirname "$WIM_FILE")
        wimlib-imagex split "$WIM_FILE" "$WIM_DIR/install.swm" 3800
        echo "Removing original oversized install.wim..."
        rm -f "$WIM_FILE"
    else
        echo "install.wim is under 4GB. No splitting required."
    fi
else
    echo "No install.wim found. Proceeding."
fi

sync

echo "====================================================="
echo "SUCCESS! Disks prepared."
echo "You can now reboot. Ensure your motherboard UEFI is set "
echo "to boot from the ISO Disk ($ISO_DEV)."
echo "====================================================="
```

Save the file (**Ctrl+O**, **Enter**, **Ctrl+X**) and execute it again:

```
./prep_windows.sh
```

It will breeze right past that formatting step this time!

Is this a concerning error

WARNING: Both disks will be COMPLETELY WIPED.

Yes, that error is fatal to the process — and I led us right into the exact trap we were trying to avoid.

Why it failed:

My previous script formatted the ISO disk as FAT32 first, and then told 7-Zip to extract the entire

of the ISO and drop it onto the FAT32 disk, the FAT32 file system flat-out rejected it. 7-Zip immediately crashed (`E_FAIL`) before the script even had a chance to reach the splitting step.

The Fix:

Instead of using 7-Zip, the script needs to mount the Windows ISO internally as a virtual CD-ROM, copy everything except the massive WIM file to the FAT32 disk, and then split the WIM file directly from the virtual CD-ROM onto the destination disk.

Here is the structurally sound version of the script. It installs `rsync` to handle the file transfer intelligently and removes the 7-Zip dependency entirely.

Open the file, delete the old contents, and paste this in:

```
nano prep_windows.sh
```

```
#!/usr/bin/env bash
set -euo pipefail

if [[ $EUID -ne 0 ]]; then
   echo "Error: This script must be run as root."
   exit 1
fi

echo "Updating repositories and installing required packages..."
apt-get update -y
# Swapped p7zip for rsync
apt-get install -y wimtools parted dosfstools util-linux rsync

echo -n "Enter 2 NVMe/SATA serial numbers separated by a space: "
read -r SERIAL1 SERIAL2

echo -n "Enter absolute path to bootable Windows ISO: "
read -r ISO_PATH

if [[ ! -f "$ISO_PATH" ]]; then
    echo "Error: ISO file not found at $ISO_PATH"
    exit 1
fi

echo -n "Enter the destination drive serial (for the Windows bootloaders): "
read -r DEST_SERIAL

get_dev_by_serial() {
    local target_serial="$1"
    lsblk -ndo NAME,SERIAL | while read -r dev sn; do
        if [[ "$sn" == "$target_serial" ]]; then
            echo "/dev/$dev"
            return
        fi
    done
}

DEV1=$(get_dev_by_serial "$SERIAL1")
DEV2=$(get_dev_by_serial "$SERIAL2")

if [[ -z "$DEV1" ]] || [[ -z "$DEV2" ]]; then
    echo "Error: Could not locate drives matching one or both provided serial numbers
    exit 1
fi

if [[ "$DEST_SERIAL" == "$SERIAL1" ]]; then
    TARGET_DEV="$DEV1"
    ISO_DEV="$DEV2"
elif [[ "$DEST_SERIAL" == "$SERIAL2" ]]; then
    TARGET_DEV="$DEV2"
    ISO_DEV="$DEV1"
else
    echo "Error: Destination serial must exactly match one of the two initial seri
    exit 1
fi

get_part_name() {
    if [[ "$1" == *[0-9] ]]; then
        echo "${1}p1"
    else
        echo "${1}1"
    fi
}

echo "-----------------------------------------------------"
echo "Target Bootloader Disk : $TARGET_DEV ($DEST_SERIAL)"
echo "ISO Installation Disk  : $ISO_DEV (Other)"
echo "Windows ISO Path       : $ISO_PATH"
echo "-----------------------------------------------------"
echo "WARNING: Both disks will be COMPLETELY WIPED."
echo -n "Type 'YES' to proceed: "
read -r CONFIRM
if [[ "$CONFIRM" != "YES" ]]; then
    echo "Operation aborted."
    exit 0
fi

# We now need two mount points: one for the USB disk, one for the ISO file
MNT_ISO_DISK=$(mktemp -d)
MNT_ISO_FILE=$(mktemp -d)

cleanup() {
    echo "Cleaning up mounts..."
    umount "$MNT_ISO_DISK" 2>/dev/null || true
    umount "$MNT_ISO_FILE" 2>/dev/null || true
    rmdir "$MNT_ISO_DISK" 2>/dev/null || true
    rmdir "$MNT_ISO_FILE" 2>/dev/null || true
}
trap cleanup EXIT

# -----------------------------------------------------
# PREPARE TARGET DISK
# -----------------------------------------------------
echo "Preparing Target Disk ($TARGET_DEV)..."
wipefs -a "$TARGET_DEV"
parted -s "$TARGET_DEV" mklabel gpt
parted -s "$TARGET_DEV" mkpart EFI fat32 1MiB 513MiB
parted -s "$TARGET_DEV" set 1 esp on
sleep 2

TARGET_PART=$(get_part_name "$TARGET_DEV")
echo "Formatting Target ESP on $TARGET_PART..."
mkfs.fat -F32 "$TARGET_PART"

# -----------------------------------------------------
# PREPARE ISO DISK
# -----------------------------------------------------
echo "Preparing ISO Disk ($ISO_DEV)..."
wipefs -a "$ISO_DEV"
parted -s "$ISO_DEV" mklabel gpt
parted -s "$ISO_DEV" mkpart primary fat32 1MiB 100%
parted -s "$ISO_DEV" set 1 esp on
sleep 2

ISO_PART=$(get_part_name "$ISO_DEV")
echo "Formatting ISO Disk as FAT32 on $ISO_PART..."
mkfs.fat -F32 "$ISO_PART"

echo "Mounting ISO Disk..."
mount "$ISO_PART" "$MNT_ISO_DISK"

echo "Mounting Windows ISO as a loop device..."
mount -o loop,ro "$ISO_PATH" "$MNT_ISO_FILE"

echo "Copying files (excluding the massive install.wim) to the ISO Disk..."
# Using rsync to copy everything except install.wim (case-insensitive exclude)
rsync -rtv --no-o --no-g --no-perms --exclude='*[iI][nN][sS][tT][aA][lL][lL].[wW][iI

echo "Searching for install.wim inside the ISO..."
WIM_FILE=$(find "$MNT_ISO_FILE" -type f -iname 'install.wim' | head -n

if [[ -n "$WIM_FILE" ]]; then
    WIM_SIZE=$(stat -c%s "$WIM_FILE")
    if [[ $WIM_SIZE -gt 4294967295 ]]; then
        echo "install.wim is larger than 4GB. Splitting into ISO Disk..."
        # Split directly from the mounted ISO to the FAT32 disk
        wimlib-imagex split "$WIM_FILE" "$MNT_ISO_DISK/sources/install.swm" 3800
    else
        echo "install.wim is under 4GB. Copying directly..."
        cp "$WIM_FILE" "$MNT_ISO_DISK/sources/"
    fi
else
    echo "No install.wim found. Proceeding."
fi

sync

echo "====================================================="
echo "SUCCESS! Disks prepared."
echo "You can now reboot. Ensure your motherboard UEFI is set "
echo "to boot from the ISO Disk ($ISO_DEV)."
echo "====================================================="
```

Run it exactly as you did before (`./prep_windows.sh`). You will see it quickly copy the smaller files, and then pause while it streams and splits the `install.wim` directly to the disk without breaking the FAT32 limits.

Previously when the datacenter mounted the ISO directly to USB the Windows installation worked normally, right now I see the error, A Windows driver your installation needs is missing,

can any part of this setup cause it

Yes, this is entirely caused by our setup method, and it is a classic quirk of how Windows Setup behaves. The good news is that you can fix it right now from that exact error screen without rebooting or starting over.

Why this is happening:

To guarantee your motherboard would boot from our custom installation disk, the script formatted it as an **EFI System Partition (ESP)**. By design, Windows aggressively protects EFI partitions by hiding them and refusing to assign them a drive letter (like `C:` or `D:`).

When you clicked "Install Now", the Windows setup engine scanned all available drive letters looking for the installation files. Because our ISO disk is hidden, Windows couldn't find the files, panicked, and threw the generic "media driver is missing" error. When the datacenter mounted the ISO, they used a Virtual CD-ROM, which Windows automatically assigns a drive letter to, bypassing this issue.

Here is how to unhide the partition and force Windows to see it.

1. **Open the Command Prompt**

   On the screen with the error message, press **Shift + F10** on your keyboard. (If you are on a laptop or certain KVM consoles, you might need to press **Fn + Shift + F10**).

   This will pop open a black command prompt window with administrator privileges.

2. **Launch Diskpart and list volumes**

   Type the following command and press Enter:

   ```
   diskpart
   ```

   Once the `DISKPART>` prompt appears, type:

   ```
   list vol
   ```

3. **Identify and select your ISO disk**

   Look at the list of volumes. You are looking for the volume that is formatted as **FAT32** and is roughly **232 GB** in size. Note its Volume Number (e.g., Volume 1, Volume 2).

   Type the following, replacing X with your actual volume number:

   ```
   select vol X
   ```

4. **Assign a drive letter**

   Force Windows to assign it a drive letter so the installer can see it:

   ```
   assign letter=D
   ```

   (Note: If D is already taken by something else, you can use E or Z).

   Type `exit` and press Enter to close Diskpart, then close the black command prompt window.

5. **Restart the Setup process**

   Click **Cancel** or the **X** on the error message to back out. This will drop you back to the main purple "Install Now" screen.

   Click **Install Now** again. Windows will scan the drives, see the D: drive we just exposed, find the installation files, and allow you to proceed to the disk selection screen.

Note: If you run `list vol` in step 2 and you do not see 232GB drives listed anywhere, let me know. That would mean Windows PE is genuinely lacking the hardware driver for the specific SATA/SAS controller on your Cherry Servers motherboard, and we will need to inject the driver.

Ask Gemini

Pro Extended

Gemini is AI and can make mistakes.

Motivational Statements for Struggling...

Tracerouteu Analysis Network Path Issues

Ian Theron